INDIAN INSTITUTE OF INFORMATION TECHNOLOGY KALYANI

Department of Computer Science and Engineering

Software Engineering Project Assignment

MediFlow

Hospital Management System

Assignment 1 · Hospital Management System

UML Diagrams

GROUP 11

Sl.	Name of the member	Registration number
1	Akash Chakraborty	CSE/23011/1059
2	Anipriyo Mandal	CSE/23017/1065
3	Arpan Mandal	CSE/23027/1075
4	Himanshu Patel	CSE/23052/1100
5	Pranjali Sirari	CSE/23072/1120

Live demo at mediflow-hms-6vw1.onrender.com

Source code at github.com/anipriyo/mediflow-hms

Submitted through Google Classroom · October 2026

Group Details

Table 1. Group and assignment details

Field	Value
Group number	11
Assignment number and title	Assignment 1, Hospital Management System
Document	UML Diagrams (document 3 of 5)
Project name	MediFlow Hospital Management System
Problem statement (brief)	Develop a system to manage patients, doctors, appointments, prescriptions, billing and medical records.
Companion documents	Software Requirements Specification (SRS), Software Design Document (SDD), Software Test Plan, Software Test Report
Live demo	mediflow-hms-6vw1.onrender.com
Source code	github.com/anipriyo/mediflow-hms

Table 2. Group members (names and registration numbers)

Sl.	Name	Registration number
1	Akash Chakraborty	CSE/23011/1059
2	Anipriyo Mandal	CSE/23017/1065
3	Arpan Mandal	CSE/23027/1075
4	Himanshu Patel	CSE/23052/1100
5	Pranjali Sirari	CSE/23072/1120

Document control

Table 3. Revision history

Version	Date	Description
0.1	15-Oct-2026	Project selected (Assignment 1, Hospital Management System)
0.2	18-Oct-2026	Scope, user roles and feature set agreed (core, hospital operations, distinctive features)
0.5	21-Oct-2026	Requirements baselined (50 FR, 10 NFR, 12 BR), architecture and UML diagrams
0.9	21-Oct-2026	Prototype implemented, unit, integration, system and acceptance tests executed
0.95	22-Oct-2026	Robustness / edge-case testing, additional defects fixed and regression-tested
1.0	22-Oct-2026	Final report for submission
1.1	22-Oct-2026	Prototype deployed online (Render), live demo link added
1.2	25-Oct-2026	Issued as a separate document (UML Diagrams), one of five

Summary

MediFlow is a web-based Hospital Management System that manages **patients, doctors, appointments, prescriptions, billing and medical records**, extended with laboratory, pharmacy and in-patient (bed and admission) management. Beyond the core requirements, it introduces a **smart queue** with tokens and waiting-time estimates, a **live bed board** with a discharge workflow tied to billing clearance, and **role-based access control with an append-only audit trail** for six staff roles.

This document collects the sixteen UML diagrams of the system (use case, class, sequence, state, activity, component and deployment) and the supporting logical data model, each with a short explanation and the requirements it covers.

> **Live demo.** The prototype runs online at mediflow-hms-6vw1.onrender.com and opens in any browser with no installation. Sign in with one of the demo accounts (admin, reception, dr.sharma, lab, pharmacy and billing, all with the password Mediflow@123). It uses a free cloud plan, so the first visit after 15 idle minutes takes about a minute to load, and the demo data is reset to fresh records dated that day whenever the server restarts.
>
> **Source code and running it locally.** The full source code is at github.com/anipriyo/mediflow-hms. Install Node.js 20 or later, run `npm install` and then `npm run dev` in the project folder, and open localhost port 5173 in a browser.

Table of Contents

Group Details..... 2
Document control..... 2
Summary..... 3
Table of Contents..... 4
UML DIAGRAMS..... 5
1. Introduction..... 5
1.1 Notation used..... 6
2. Use Case Diagrams..... 6
3. Class Diagrams..... 10
4. Sequence Diagrams..... 13
5. State Diagrams..... 17
6. Activity Diagram..... 19
7. Component Diagram..... 20
8. Deployment Diagram..... 21
9. Logical Data Model..... 22

UML DIAGRAMS

> All diagrams follow UML 2.5 and were drawn as text with PlantUML. The sources are kept with the code in `docs/uml/*.puml` and rendered to `docs/uml/png/`, so every diagram can be regenerated and reviewed like source code. The same diagrams are explained in their design context in the Software Design Document.

1. Introduction

This document collects the sixteen UML diagrams of MediFlow, plus the logical data model that supports them. They describe the system from four viewpoints. **Use case diagrams** show who uses the system and for what, **class diagrams** show its static structure, **sequence, state and activity diagrams** show its behaviour, and **component and deployment diagrams** show how it is built and installed.

Table 4. Diagram catalogue

No.	Diagram	Type	What it shows	Covers	Source (.puml)
1	System overview	Use case	Six staff actors and the nine major functions of the system (context level)	All modules	01a-use-case-overview
2	Outpatient care	Use case	Registration, booking, check-in, smart queue, consultation, laboratory and pharmacy use cases with «include» and «extend»	FR-07 to FR-12, FR-15 to FR-37	01b-use-case-outpatient
3	Inpatient care and billing	Use case	Bed board, admission, transfer, discharge with billing clearance, invoices and payments	FR-38 to FR-47	01c-use-case-inpatient-billing
4	Administration and access control	Use case	Login with lock-out, dashboards, users and roles, reports and audit trail	FR-01 to FR-06, FR-13, FR-14, FR-48 to FR-50	01d-use-case-admin
5	Domain model	Class	Entities with attributes, operations, associations and multiplicities	Data of all FRs	02-class-domain
6	Service layer design	Class	React client classes, Express router and middleware, services, rule modules and data access, with dependencies	NFR-06 (maintainability)	03-class-design
7	Book an appointment	Sequence	Slot check, conflict detection and transactional booking, with the slot-taken alternative	FR-15 to FR-17	04-seq-booking
8	Check-in, queue and consultation	Sequence	Token and ETA calculation, calling the next patient, consultation, prescription, lab order and charges	FR-21 to FR-29	05-seq-queue-consultation
9	Admission and discharge	Sequence	Bed allocation, discharge request, billing clearance and the administrator override	FR-39 to FR-46	06-seq-admission-discharge
10	Login and access control	Sequence	Password check, lock-out after five failures, token issue and permission check on each request	FR-01 to FR-04	07-seq-login
11	Appointment	State machine	Life cycle of an appointment from booking to completion, cancellation or no-show	BR-04	08-state-appointment
12	Bed	State machine	Available, reserved, occupied, cleaning and maintenance states with their allowed transitions	BR-08	09-state-bed
13	Admission, invoice and lab order	State machine	Three composite life cycles in one diagram	BR-06, BR-09, BR-10	10-state-admission-invoice-lab
14	Outpatient journey	Activity	One patient visit across five swimlanes (reception, doctor, laboratory, pharmacy, billing)	Outpatient FRs	13-activity-opd
15	Layered architecture	Component	Client, application and data tiers, and the four server layers	NFR-06, NFR-10	11-architecture
16	Deployment	Deployment	Workstations, application server with Node.js runtime and SQLite file, and printers	NFR-07, NFR-09	12-deployment

No.	Diagram	Type	What it shows	Covers	Source (.puml)
17	Logical data model	Entity relationship (supporting)	The 20 database tables, keys and relationships in crow's-foot notation	Data of all FRs	14-er-diagram

1.1 Notation used

- **Use case diagrams.** Stick figures are actors (staff roles), ovals are use cases and the rectangle is the system boundary. «include» marks behaviour that always happens as part of the base use case, while «extend» marks optional behaviour.
- **Class diagrams.** Each class shows its attributes and operations. Lines show associations with multiplicities, a filled diamond marks composition and an arrow marks a dependency. Stereotypes such as «lib/api.js» name the source file that implements a class.
- **Sequence diagrams.** Lifelines run from left (actor and user interface) to right (database). Messages are numbered, and alt and opt fragments show the error and override paths that are also covered by test cases.
- **State and activity diagrams.** Rounded boxes are states or actions, the filled circle is the start and the ringed circle is the end. Transitions that are not drawn are rejected by the system with HTTP 409.

2. Use Case Diagrams

The system has six staff actors (Receptionist, Doctor, Lab Technician, Pharmacist, Billing Staff and Administrator). Patients are external to the system, because they do not log in and their data is entered by staff. A single diagram with six actors and about forty use cases would be unreadable, so the model is split into an overview and three detailed diagrams.

Use Case Diagram 1 - System overview (context level)

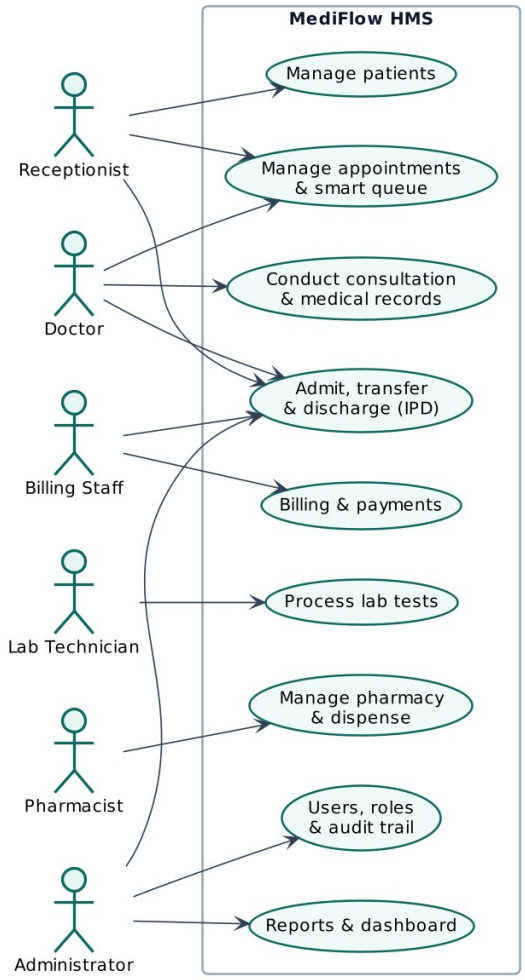


Figure 1. Use case diagram (system overview)



Figure 2. Use case diagram (outpatient care)

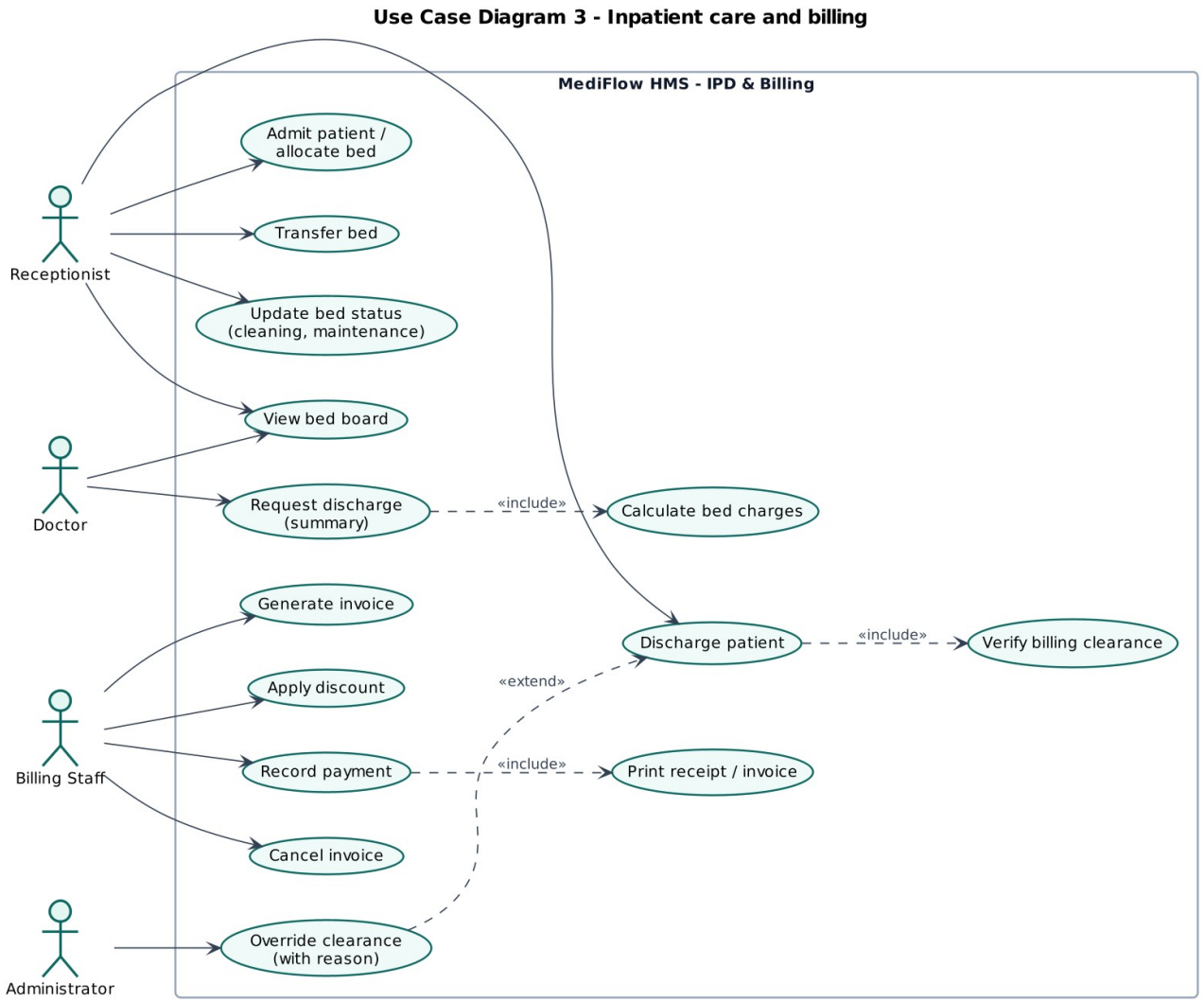


Figure 3. Use case diagram (inpatient care and billing)

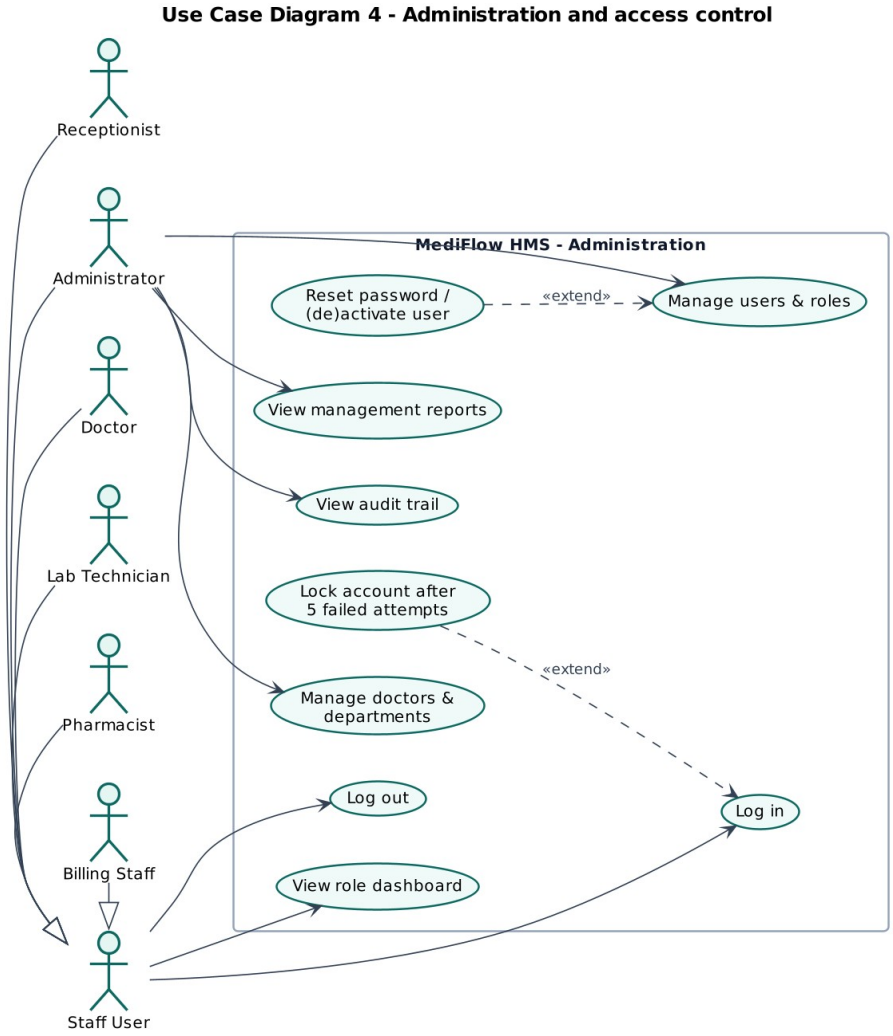


Figure 4. Use case diagram (administration and access control)

Detailed specifications of the five most important use cases (book appointment, check in and manage queue, conduct consultation, discharge patient and record payment) are given in the Software Design Document, Section 5.1.

3. Class Diagrams

The **domain class diagram** shows the main entities, their attributes, operations and associations with multiplicities. A patient books many appointments, and each appointment has at most one consultation, which may produce one prescription (composed of items referencing medicines) and several lab orders. Wards are composed of beds, and an admission is composed of one or more bed stays (one per bed occupied), which is how transfers and bed-day billing are modelled. Charges aggregate into invoices, which own payments.

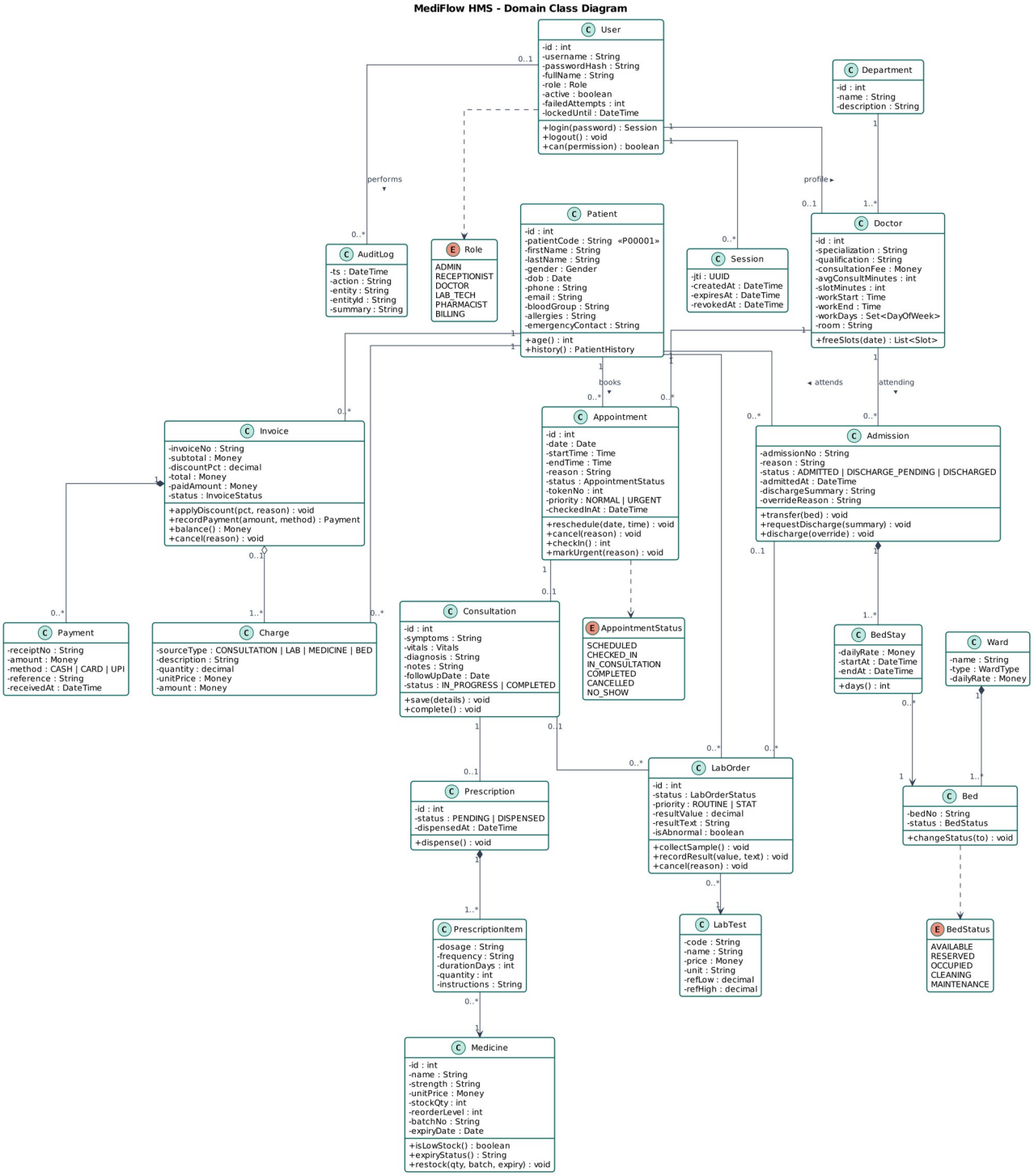


Figure 5. Domain class diagram

The **design class diagram** shows how the implementation realises the domain. Controllers delegate to services, services use pure rule modules and the data-access layer, and the cross-cutting services (Charge and Audit) are shared by all modules.

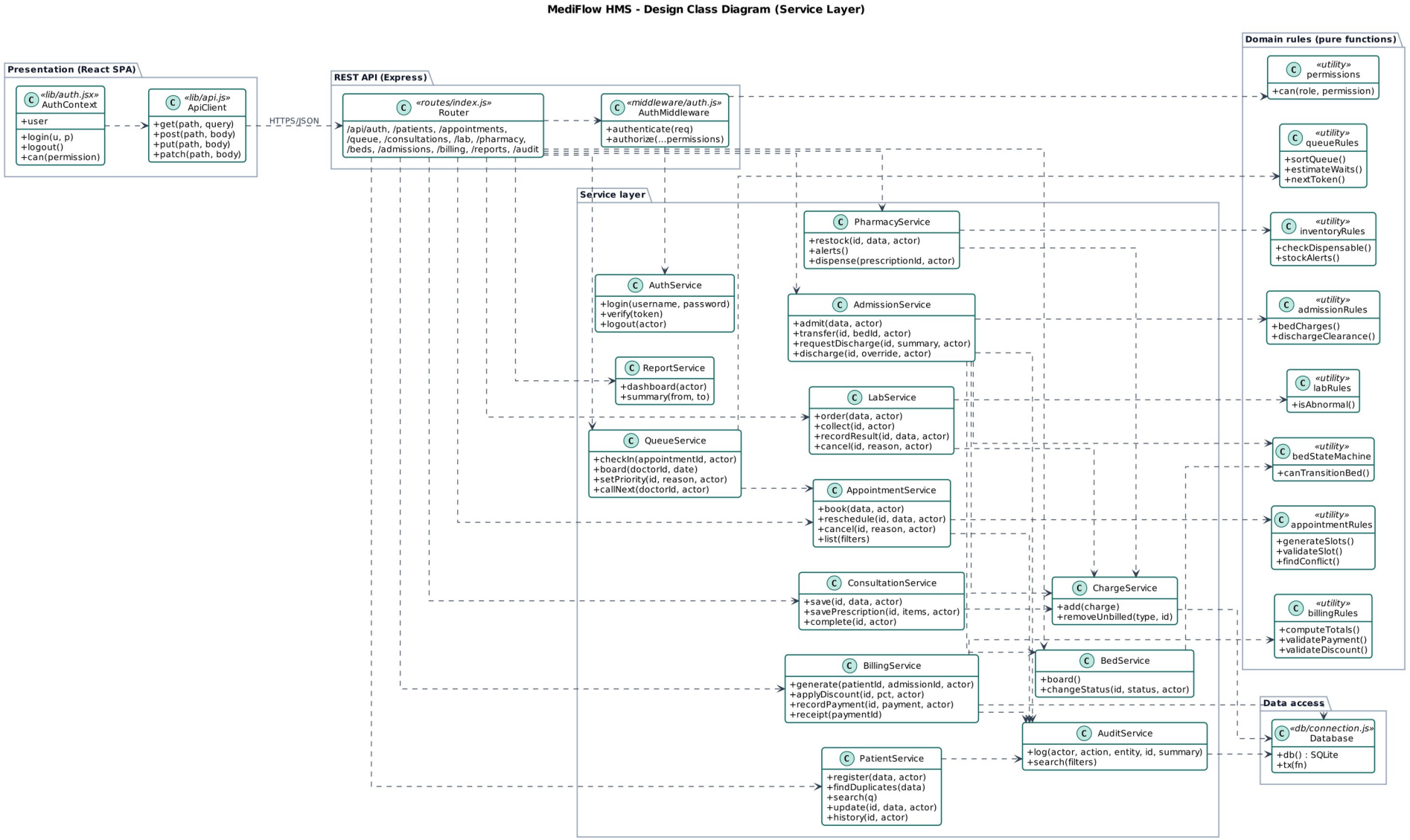


Figure 6. Design class diagram (service layer and dependencies)

4. Sequence Diagrams

The sequence diagrams show the interaction between the actors, the React user interface, the API layer, the services, the rule modules and the database for the four most important scenarios.

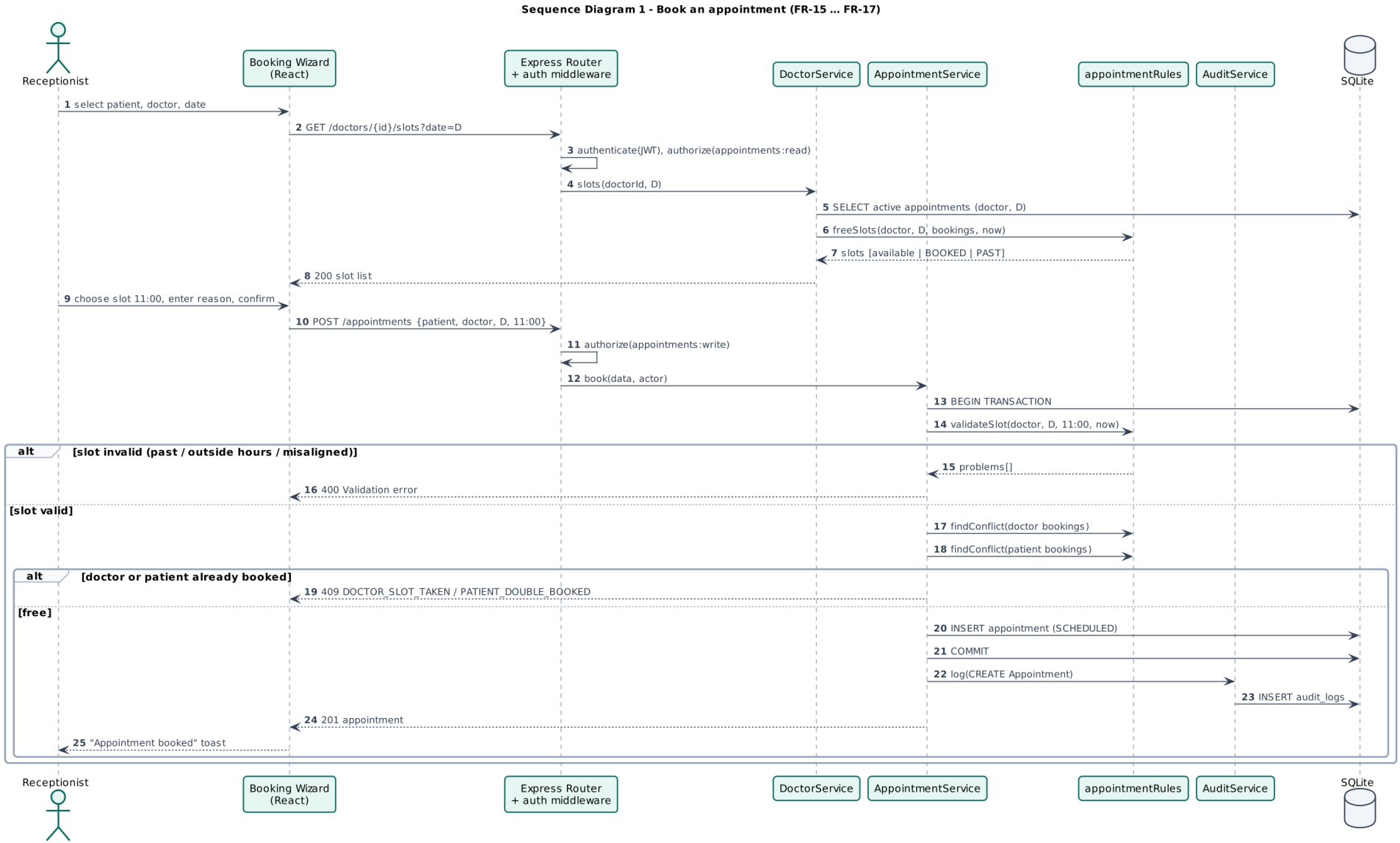


Figure 7. Sequence diagram (book an appointment)

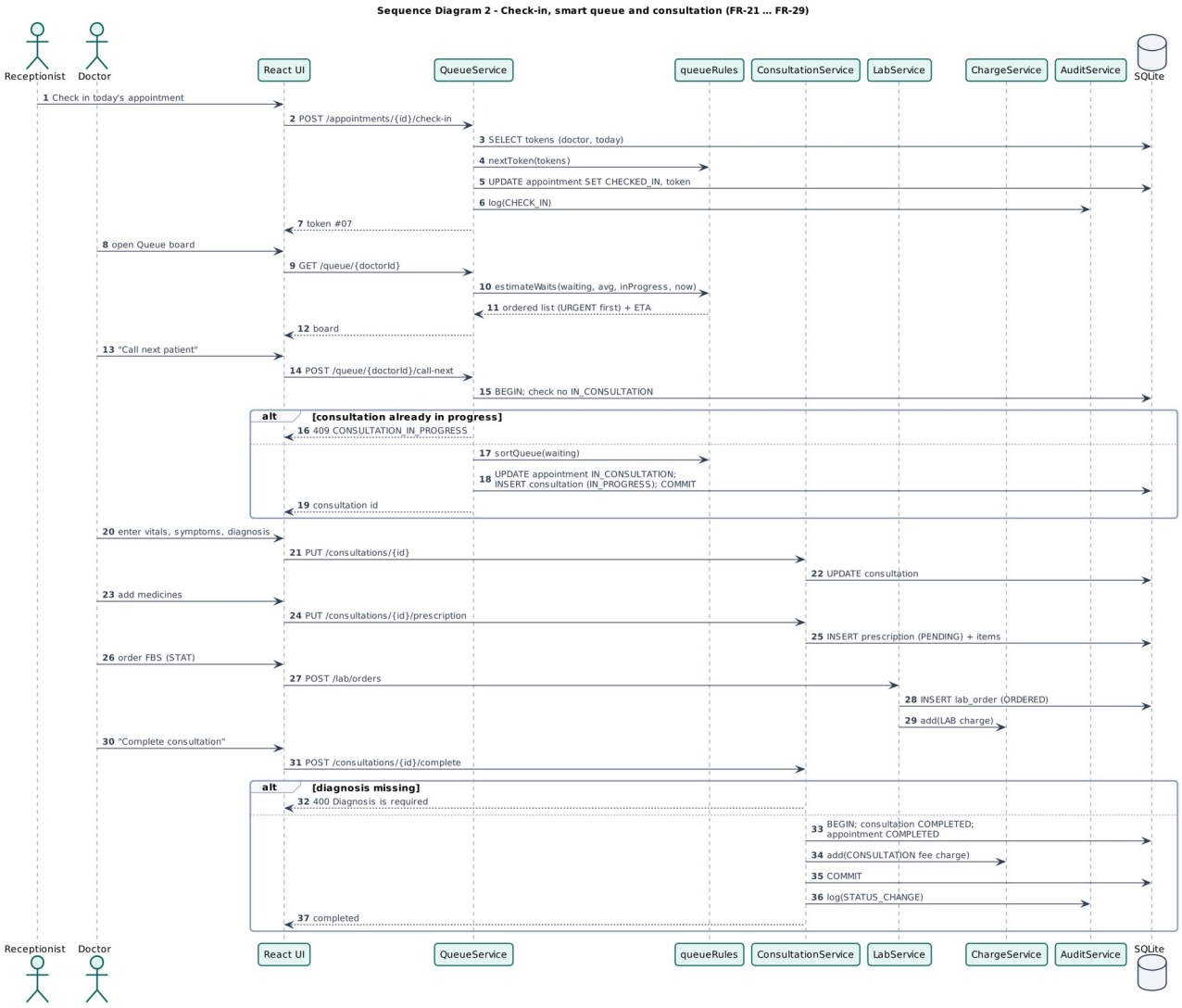


Figure 8. Sequence diagram (check-in, queue and consultation)

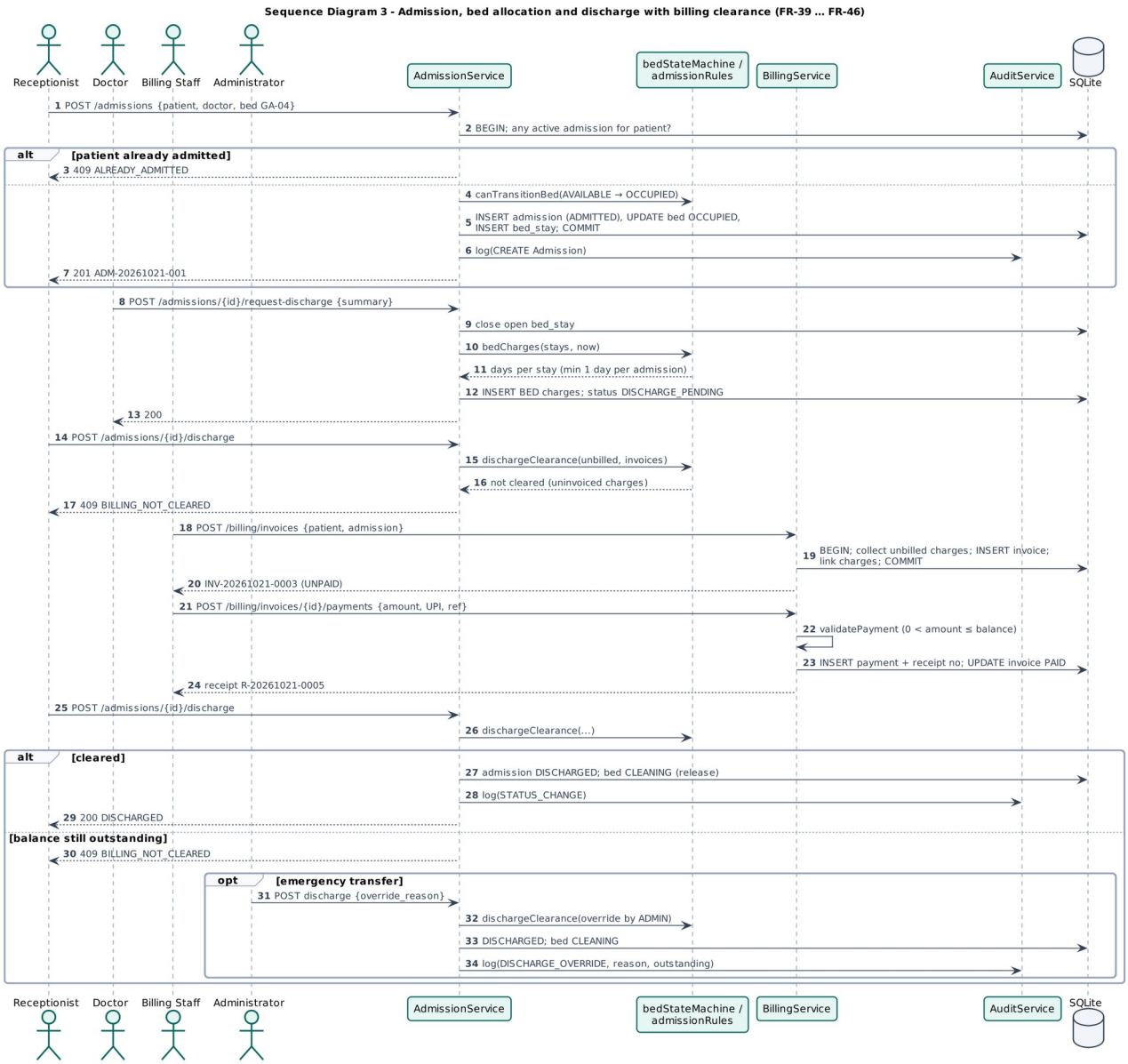


Figure 9. Sequence diagram (admission and discharge with billing clearance)

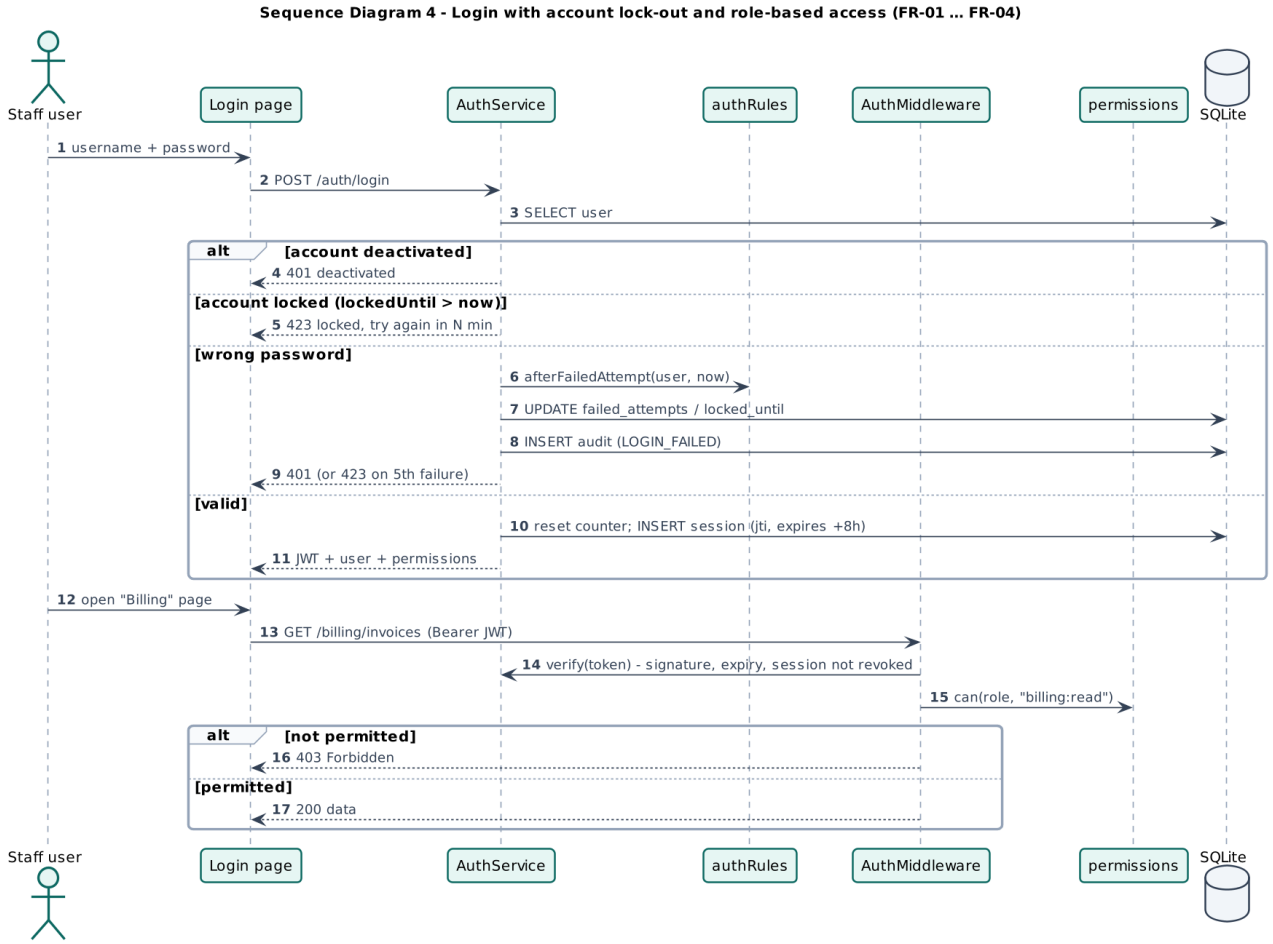


Figure 10. Sequence diagram (login, lock-out and role-based access)

5. State Diagrams

Every entity with a life cycle has a state diagram. The same transition tables are implemented in `server/domain/*` and verified by the unit tests UT-APT-10, UT-IPD-01, UT-IPD-02, UT-IPD-04, UT-LAB-01 and UT-BIL-06, so the diagrams and the code cannot drift apart unnoticed.

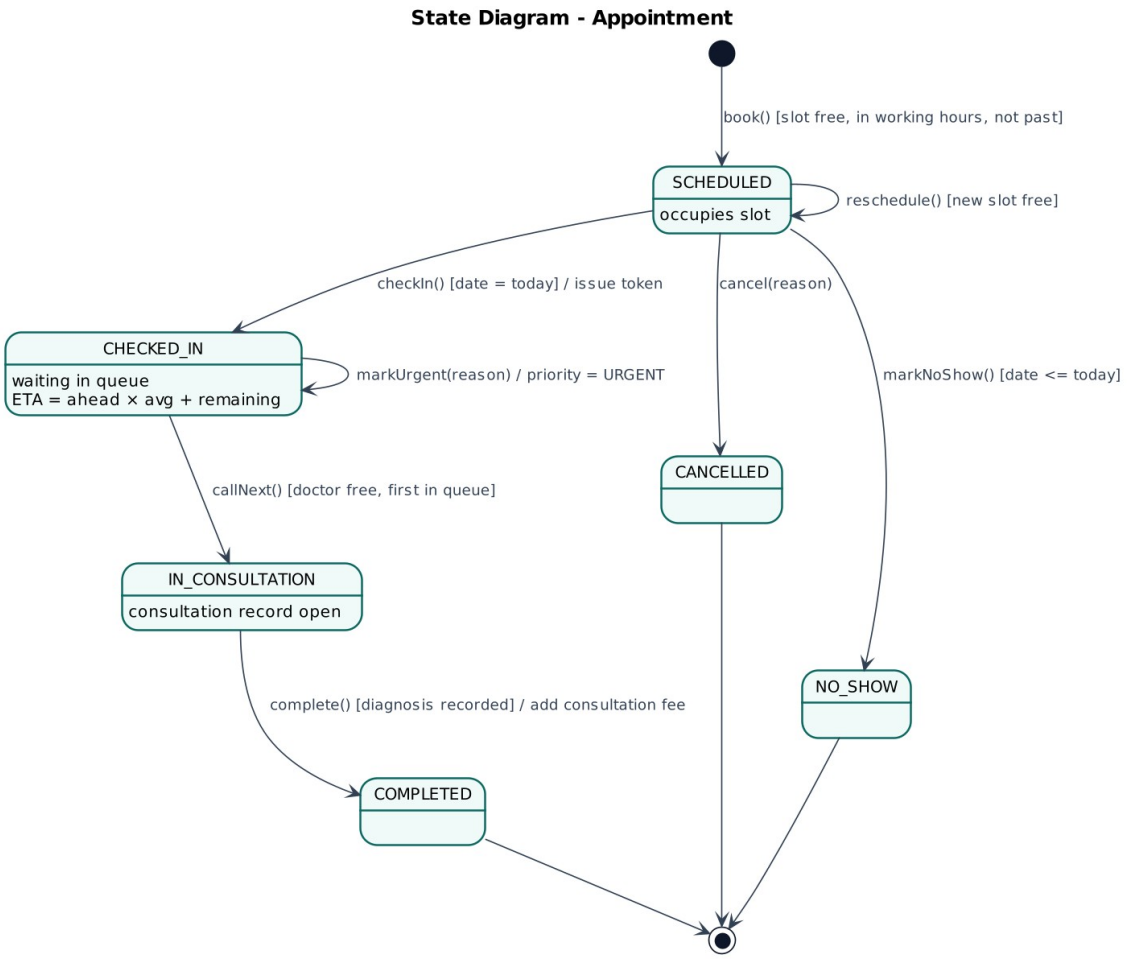


Figure 11. State diagram (Appointment)

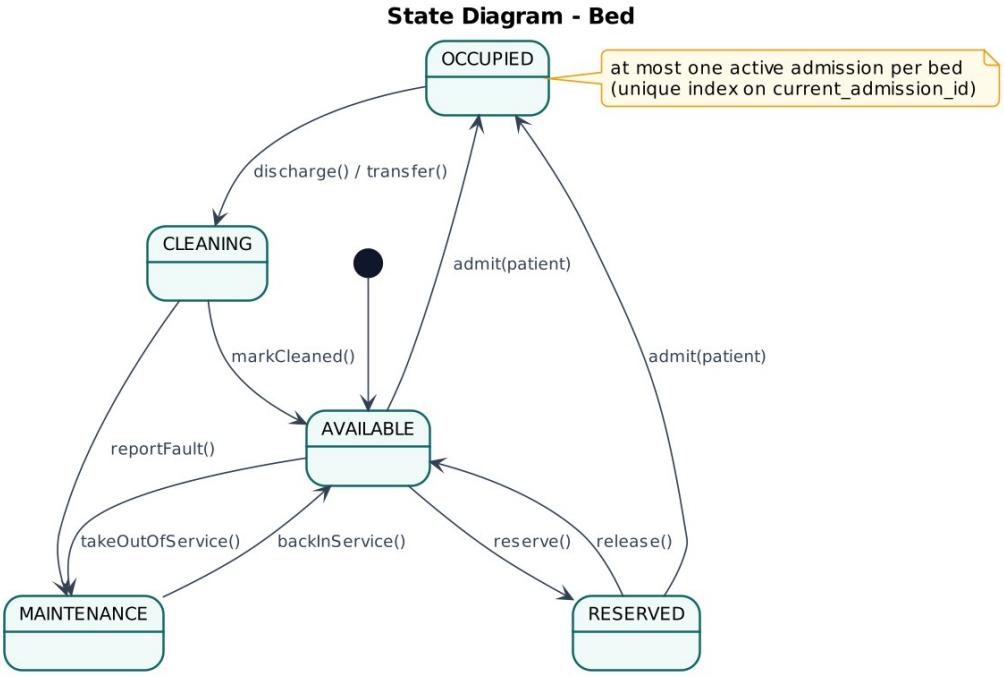


Figure 12. State diagram (Bed)

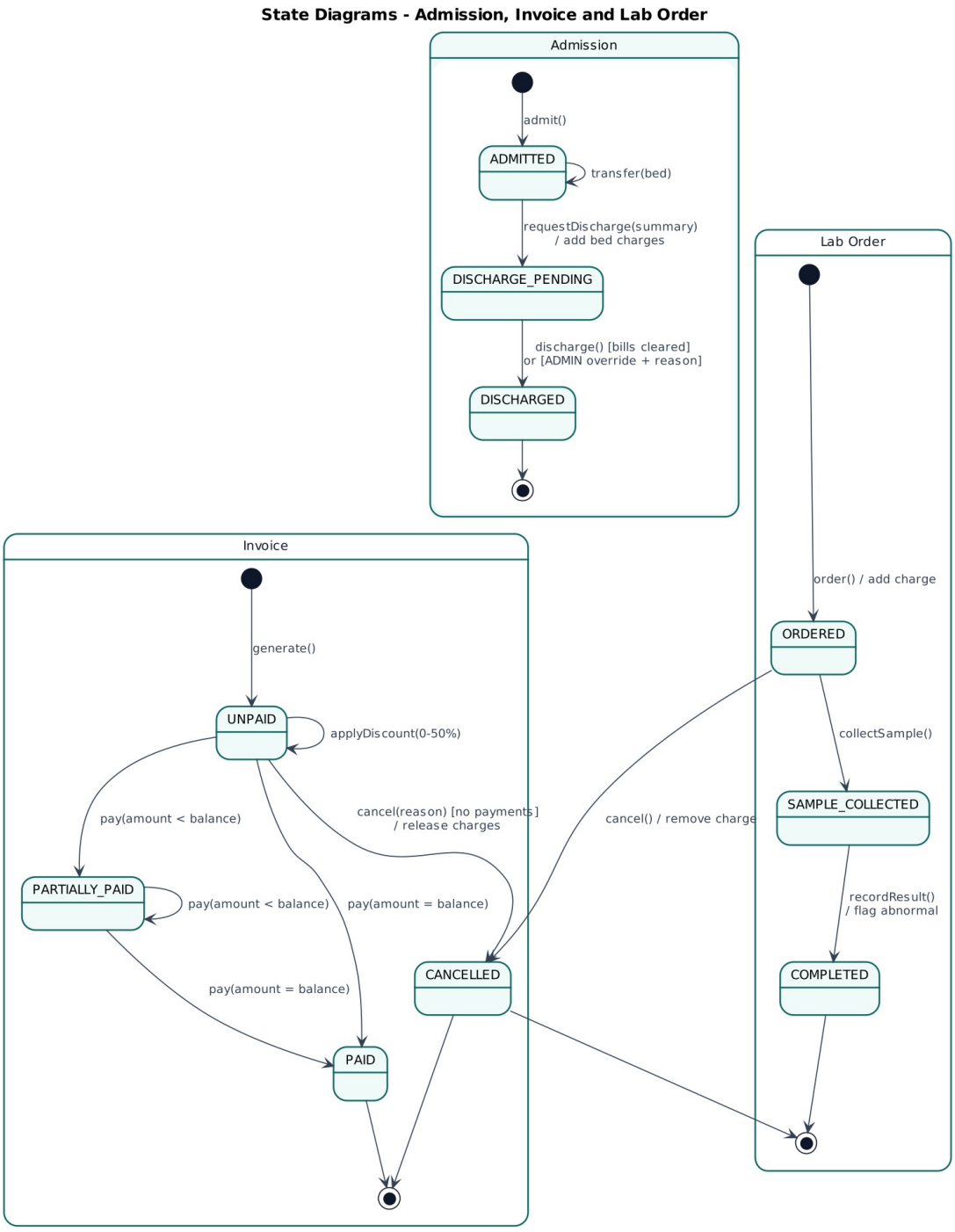


Figure 13. State diagrams (Admission, Invoice and Lab order)

6. Activity Diagram

The activity diagram complements the state diagrams by showing how responsibility for one outpatient visit passes between the roles, from registration and check-in through consultation, laboratory and pharmacy to billing.

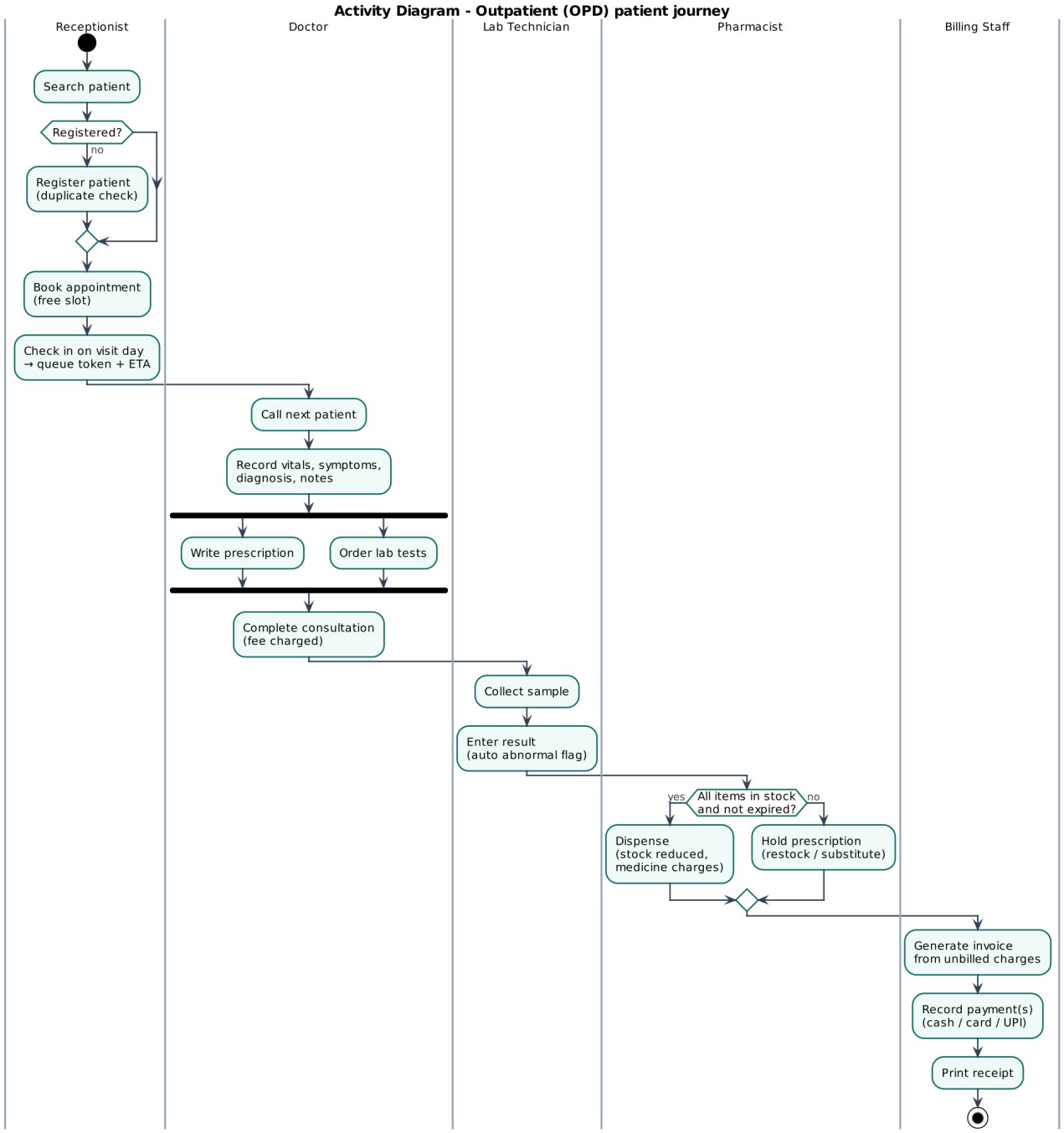


Figure 14. Activity diagram (outpatient journey across roles)

7. Component Diagram

MediFlow uses a three-tier, layered client and server architecture. The presentation tier is a React single-page application in the browser, the application tier is a Node.js and Express server with a REST API, and the data tier is an embedded SQLite database. Inside the server each layer may only call the layer below it.

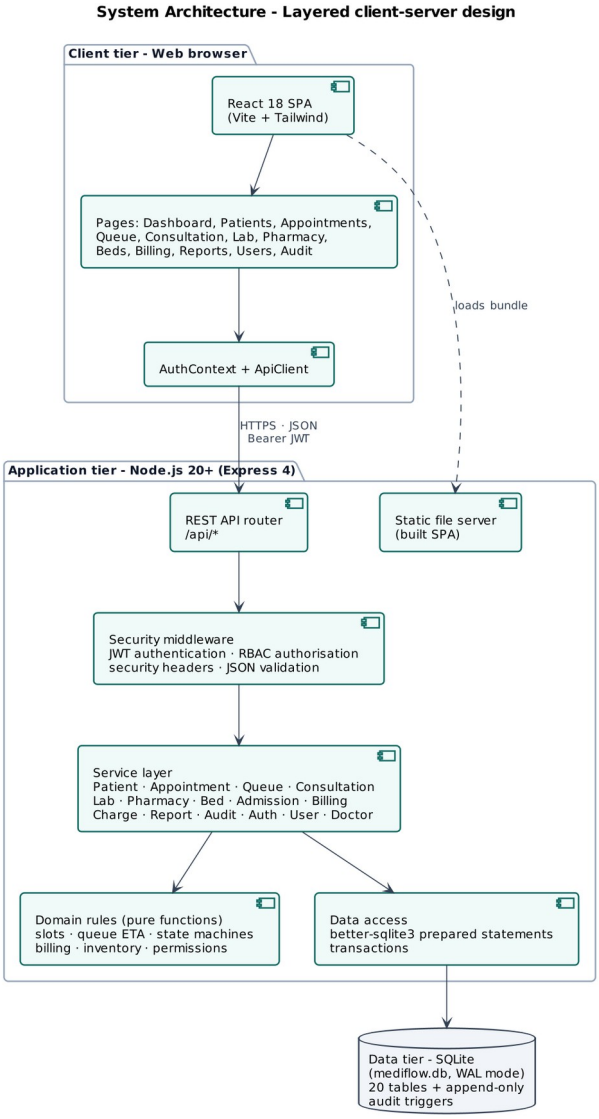


Figure 15. Component diagram (layered system architecture)

8. Deployment Diagram

In a hospital, the server runs on one application machine on the local network and staff use any browser on a workstation or tablet. Invoices and receipts are printed from the browser.

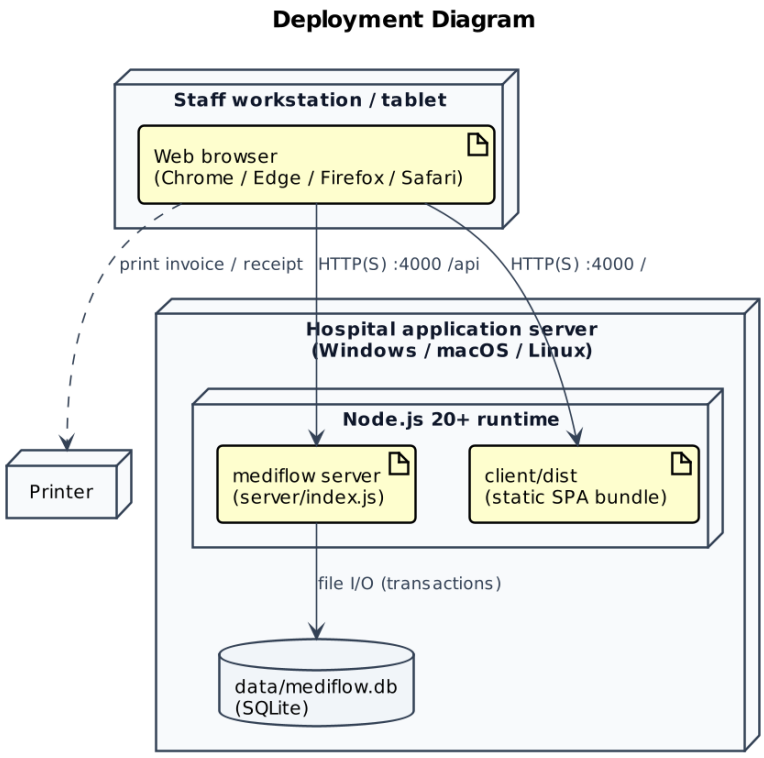


Figure 16. Deployment diagram

The public demonstration uses the same single-server design. One Node.js web service on Render's cloud serves both the built React app and the REST API with the embedded SQLite database, at mediflow-hms-6vw1.onrender.com.

9. Logical Data Model

The logical data model is not a UML diagram, but it is included because the domain class diagram maps directly onto it. It has 20 tables in third normal form, and status columns are constrained by CHECK clauses that mirror the state diagrams in Section 5.

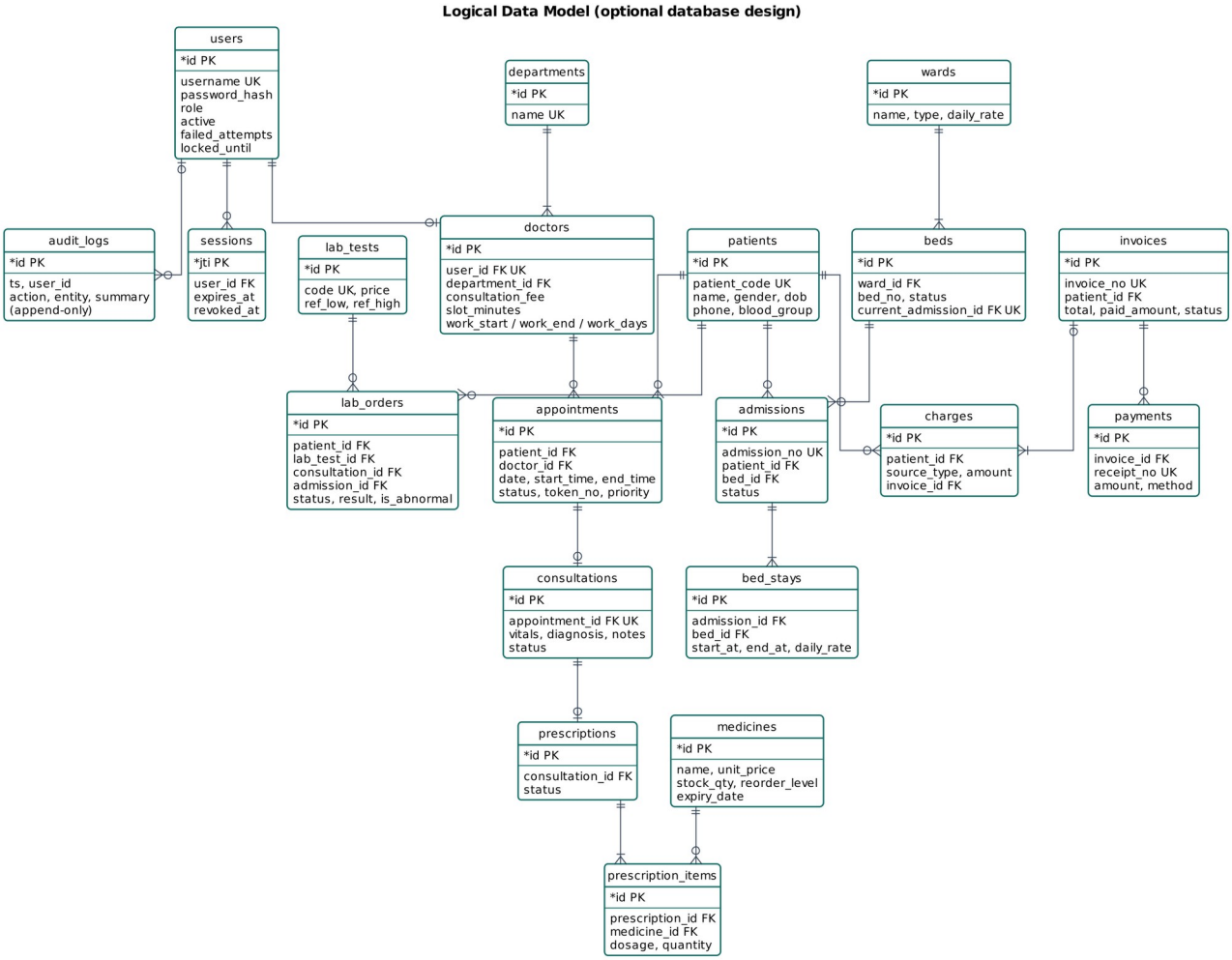


Figure 17. Logical data model (crow's-foot notation)